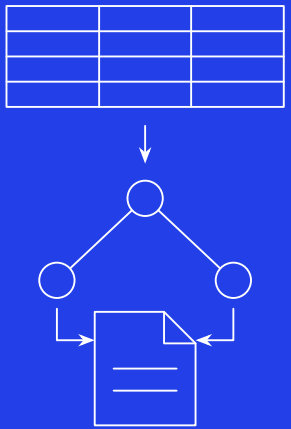
WEEK 1 / LAB 02

MLOps: Machine Learning Operations

Data Validation, Chronological Split & Baseline Tracking

Validate data. Compare models. Preserve evidence.



By the end, a peer can inspect your result

Three observable outcomes – not a promise of a better model.

1 **Protect the data and evaluation boundary**

Reject invalid rows; keep the canonical CSV unchanged and the test period held.

2 **Compare two predictors fairly**

Fit on training rows only; measure both MAEs on identical validation rows.

3 **Retrieve the evidence without the author's memory**

Find the runs, reload the saved model and explain the unchanged-config repeat.

Keep this lab local, bounded and evidence-first

Lab 02 shares the remaining Week 1 class time; it is not another four-hour meeting.

CORE TASK

Validate & split

Training mean + Random Forest
Local MLflow + save/reload
Focused tests + peer review

NOT THIS LAB

No extra platform to build

No API, container or cloud
No registry or rollback
No tuning or test-set scoring

Use the instructor’s checkpoints; record unfinished work honestly.

Complete narrow TODOs, not a platform from scratch

Use the instructor-supplied Lab 2 extension in your existing project repository.

SUPPLIED SCAFFOLD	YOUR BOUNDED WORK
Canonical CSV + manifest + approved lock	Inspect identity; validate; preserve bytes.
Loading, CLI, split, train and tracking scaffolds	Implement the specified rules and logging.
Public tests + existing PR check	Test, create two faults and review evidence.

PART 1

Validate the data. Protect the split.

A believable score starts before the model is fitted.

CHECKPOINT A

Preserve source bytes

Enforce the contract

Reserve the test period

Start only from a verified, matching Lab 2 package

Commands run in the supplied bike-demand project root, not this slide workspace.

PROJECT ROOT / SETUP + PREFLIGHT

```
uv sync --locked
uv run --locked python -m bike_demand.preflight
git switch -c lab02/data-baseline
```

Confirm the data, manifest and lock before changing code.

Missing package or setup blocked for five minutes? Ask for the supported recovery route.

Install MLflow before training

The matching Lab 2 lock must include MLflow; uv installs its approved version.

Missing MLflow? Request the approved Lab 2 package; do not use global pip.

BEFORE TRAINING / INSTALL + CHECK IN THE PROJECT ROOT

```
uv sync --locked
uv run --locked mlflow --version
```

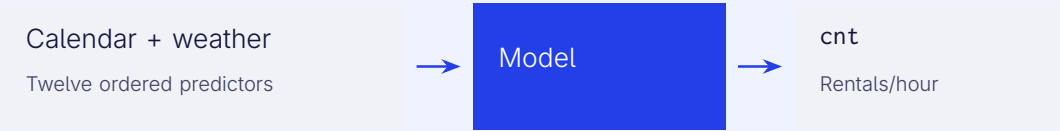
AFTER TRAINING / SECOND TERMINAL, SAME PROJECT ROOT

```
uv run --locked mlflow server \
  --backend-store-uri sqlite:///tracking/mlflow.db \
  --host 127.0.0.1 --port 5000
```

Open `http://127.0.0.1:5000`; select the experiment and compare both runs.

Estimate demand given calendar and weather inputs

This is conditional hourly demand – not a forecast of unknown future weather.



`cnt = casual + registered`

Both components reveal the answer. Keep them out of the feature matrix.

Keep exactly twelve predictors, in this order

Raw CSV fields and model inputs are different contracts.

1–4	`season`, `yr`, `mnth`, `hr`
5–8	`holiday`, `weekday`, `workingday`, `weathersit`
9–12	`temp`, `atemp`, `hum`, `windspeed`

Exclude `cnt`, `casual`, `registered`, `instant`, `dteday` from every feature matrix.

Reject invalid data before training starts

Parse declared types; name the failing field or rule. Never silently fill, clip or drop.

Categories	`season`: 1–4; `yr`, `holiday`, `workingday`: 0 or 1 `mnth`: 1–12; `hr`: 0–23; `weekday`: 0–6; `weathersit`: 1–4 All category values must be integers.
Weather values	`temp`, `atemp`, `hum`, `windspeed`: finite numbers in [0, 1].
Rows and schema	Required columns/values present; parseable dates. Unique positive integer `instant`; nonnegative integer counts.

Preserve `data/raw/hour.csv`; test invalid copies instead.

Train on the past; reserve the later test period

Course date intervals are inclusive. No shuffle and no random train/test split.

Train

01 Jan–31 Dec 2011

Fit mean and Random Forest

Validation

01 Jan–30 Jun 2012

Evaluate both

Test

01 Jul–31 Dec 2012

Held; not evaluated

Reject dates outside these intervals; do not silently discard rows.

Every row needs one partition and a stable order

Complete the split TODOs and focused tests before fitting either predictor.

1 Sort by `(dteday, hr, instant)`

Parse dates; use `instant` to break ties after date and hour.

2 Assert nonempty, disjoint and complete membership

Test inclusive boundaries; every canonical row belongs to exactly one interval.

3 Record observed train/validation counts and end keys

First and last `(dteday, hr, instant)` – not full-calendar hour totals.

PART 2

Fit two predictors. Keep their evidence.

Keep the same inputs and make both results identifiable.

CHECKPOINT B

Same validation rows

Two actual local runs

Save, reload and repeat

A feature-dependent model needs a fair comparator

Both predictors fit on train only and are evaluated on the same validation rows.

NAIVE BASELINE

Training-label mean

$\widehat{y} = \overline{y}_{\text{train}}$ for every validation row

No calendar/weather dependence.
Never average validation or test labels.

FEATURE-DEPENDENT BASELINE

Random Forest

$\widehat{y} = f(X)$ using the twelve predictors

Fit on the training matrix only.
Keep the supplied configuration fixed.

Use the course configuration without tuning

These are proposed teaching defaults, not a measured accuracy claim.

PYTHON SETTINGS – COURSE DEFAULTS, NOT A MEASURED RESULT

```python
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(
    n_estimators=50,
    max_depth=10,
    random_state=42,
    n_jobs=1,
)
```

Train rows only

Twelve ordered columns.
No extra feature search.

Other defaults

Use the locked library.
Do not change the config.

Measure absolute error on identical validation rows

Lower MAE is better for this fixed comparison; no target score is prescribed.

$$\mathrm{MAE} = \frac{1}{n_{\mathrm{val}}} \sum_{\text{validation rows}} |y - \widehat{y}|$$

Units: rentals/hour

Record both measured MAEs and explain the comparison.
Do not invent a win, a run ID or a passing threshold.

Commit training code before capturing its identity

First commit the completed code/config on your task branch; then validate and run.

EXPECTED STARTER COMMANDS – CAPTURE ACTUAL OUTCOMES

```
uv run --locked python -m bike_demand.validate --data data/raw/hour.csv
git rev-parse HEAD
uv run --locked python -m bike_demand.train --config configs/baseline.yaml
```

Code commit + canonical data ID + config + feature order

These identify the training inputs. A later report commit is a different snapshot.

Two local runs must point back to the same inputs

Use the supplied helper: SQLite stores run metadata; local files store artifacts.

RUN A – NAIVE

Actual run ID + MAE

Training mean; train/validation bounds.

RUN B – RANDOM FOREST

Actual run ID + MAE

Four settings; model version; feature order.

Read back

Both runs: training commit, canonical data ID, counts, bounds, predictors and settings.

Backend: `sqlite:///tracking/mlflow.db` Artifacts: `tracking/artifacts`

Reload the saved model, not just its screenshot

Link the local outputs to their actual run and check the returned predictions.

REQUIRED OUTPUT	OBSERVABLE CHECK
`artifacts/model.joblib`	Reload; predict finite values; recompute validation MAE.
`artifacts/model-metadata.json`	Match model version, run, training commit and data ID.
`reports/metrics.json`	Match both run IDs and MAEs.

Retain first-run evidence before repeating unchanged

Repeat the same training command once; record what actually changes.

First execution

Record both run IDs and both MAEs.
Keep the training mean and settings.

Unchanged-config repeat

Record the new run IDs and MAEs.
Compare inputs, metrics and readback.

Fixed output paths may be overwritten.
Run-specific artifacts and recorded IDs preserve each execution.

PART 3

Try to break it. Then let a peer check.

A green result is stronger when you can explain what fails and why.

CHECKPOINT C

Two transparent faults

Review the current PR

Week 1 handover

TEAM CHALLENGE – USE TINY TRAINING-PERIOD COPIES

Can your validator reject the right fault?

Start from valid rows.
Create two separate invalid copies.
Assert the specific reason for rejection.

Record actual messages.
Revalidate the unchanged canonical CSV.

FAULT A – MISSING COLUMN

Remove `temp`.

Expected reason:
missing required field.

FAULT B – OUT-OF-RANGE VALUE

Set `hum = 1.2`.

Keep all columns.
Expected reason: humidity range.

Focused tests protect rules before the full suite runs

Valid input passes; faults fail; partitions and predictor exclusions remain correct.

TARGETED CHECKS, THEN SUPPLIED SUITE, THEN CANONICAL VALIDATION

```
uv run --locked pytest -q tests/test_validate.py \
  tests/test_split.py tests/test_train.py
uv run --locked pytest -q
uv run --locked python -m bike_demand.validate --data data/raw/hour.csv
```

Also test the train-only mean and stable split order.
Local success does not prove the PR check passed for the latest pushed commit.

SWAP DRIVER AND REVIEWER

Can a peer retrieve one metric by run ID?

No verbal hints from the latest driver.
Find the saved model's run.
Read back its validation MAE.

Record a finding, response and next owner.
Merge only after review and current checks.

REVIEW THE CURRENT REVISION

1 Read the diff

No target leakage; correct split.

2 Retrieve evidence

Runs, settings and reloaded artifact.

3 Refresh PR checks

Latest pushed commit, not an older one.

The team worksheet records observations, not guesses

Complete the supplied blank worksheet as `reports/lab-02.md` in the project repo.

1. **Data and identities**
   Training commit, data ID, config/features, actual train/validation counts and bounds.

2. **Experiments and readback**
   Mean, both models' first/repeat MAEs and run IDs; reload and comparison observations.

3. **Checks, contributions and remaining limits**
   Fault/test outcomes, review response, assistance used, next action and owner.

Week 1 has a team checkpoint and one personal report

Keep project evidence and individual report submission on their separate routes.

TEAM PROJECT REPOSITORY

`m1` – ungraded

After both Week 1 labs are reviewed.
Annotated tag + short checks/blockers note.
Team code keeps peer-reviewed PRs.

YOUR PRIVATE REPORT REPOSITORY

`reports/week-1.md` – 10%

One report covers Labs 01–02.
Commit directly to personal `main`.
No report PR, CI, tag or Release.

Personal report: Sunday 23:59, using the published date and course time zone.

EXIT CHECK – EXPLAIN YOUR OWN EVIDENCE

Believable inputs. Fair comparison. Traceable results.

Why does the mean use train labels only?
Why does the test period stay held?
How does a peer find your model's actual run?

Leave with evidence, an honest limitation and a named next owner.

DATA & SPLIT

Validated source; held test.

BASELINES & TRACKING

Two runs; reload; repeat.

NEXT – WEEK 2

Locked environment and container workflow.